IMEN574 / IDSC574 · PROGRAMMING FOR AX

16

Training and Generalization

Sangwoo Mo · POSTECH
Spring 2027

Where We Are

01 **Programming with AI**

Lectures 2–4

02 **Python and Classical Machine Learning**

Lectures 5–13

03 **Deep Learning and Foundation Models**

Lectures 14–22

Outline: Training and Generalization

01 **Generalization Evidence**

Read the data boundary

02 **Optimization Diagnostics**

Control update behavior

03 **Regularization and Experiments**

Test controlled changes

01 · Generalization Evidence

01 **Generalization Evidence**

Read the data boundary

02 **Optimization Diagnostics**

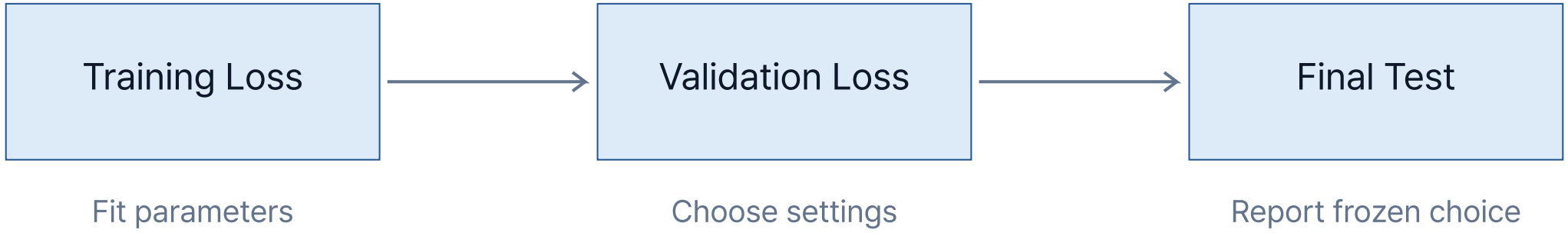
Control update behavior

03 **Regularization and Experiments**

Test controlled changes

Training Evidence

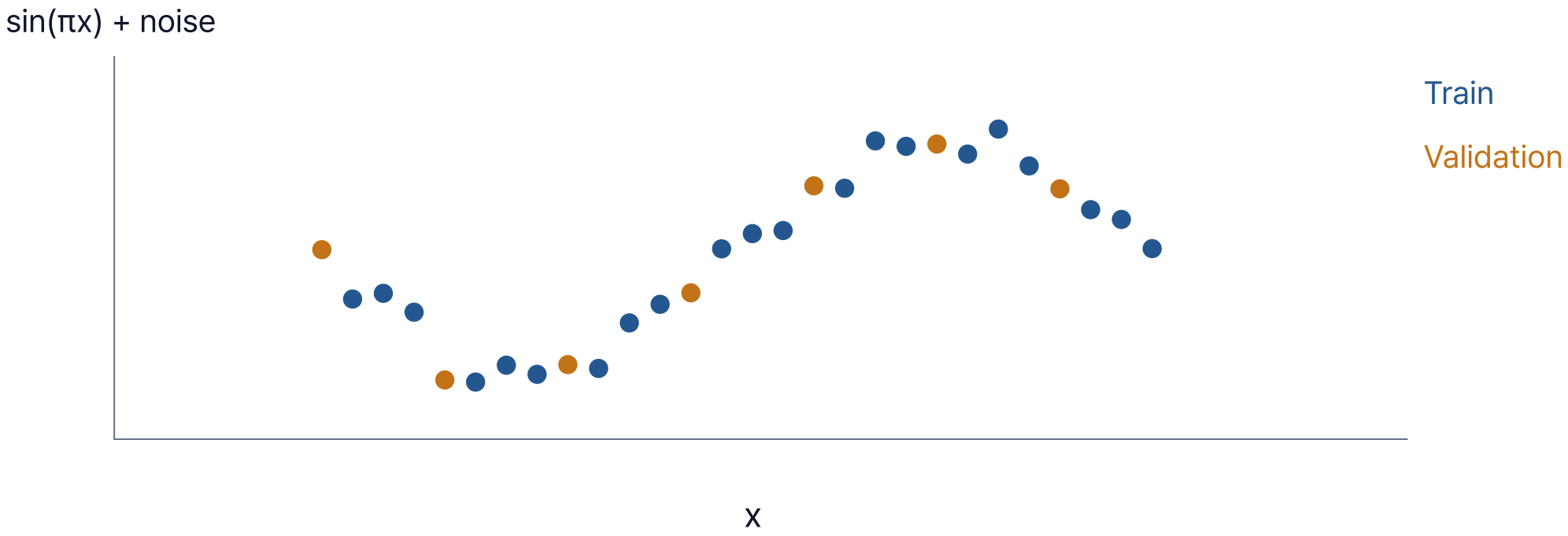
Separate optimization progress from generalization evidence



Each dataset has a different decision role

A Controlled Noisy Dataset

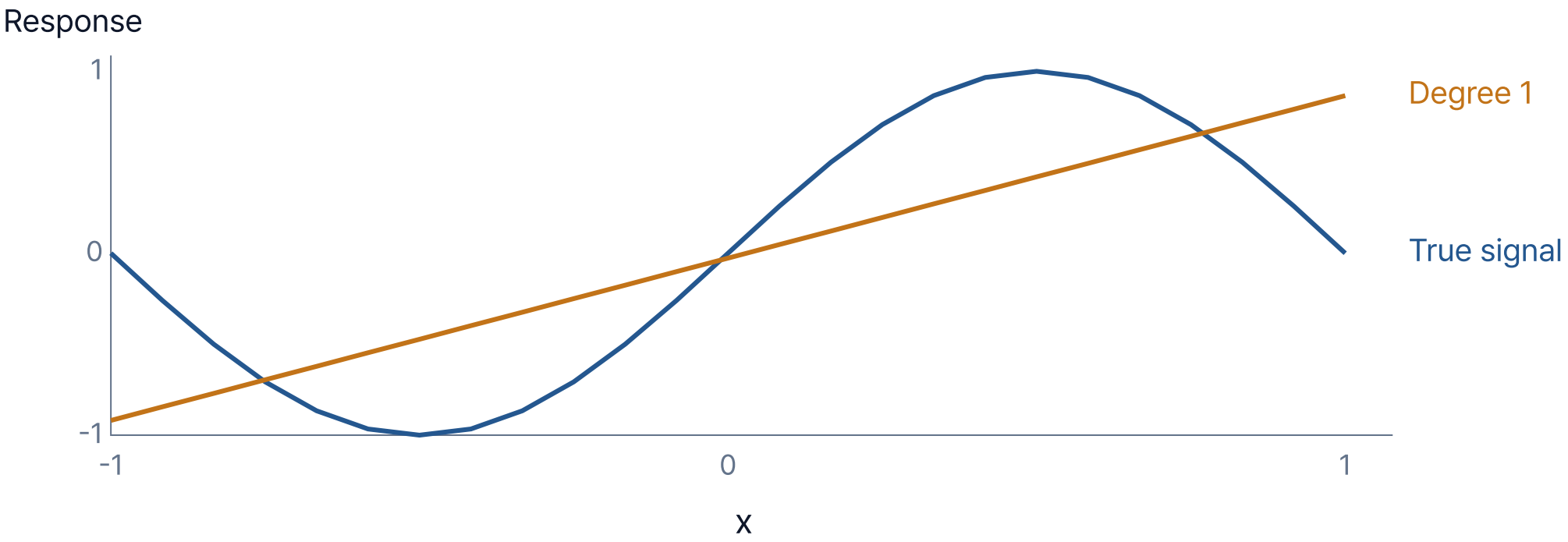
Use known signal plus noise to expose fitting behavior



The held-out points were separated before model comparison

Underfitting Evidence

A simple family can miss systematic structure



The linear family cannot follow both bends

Capacity and Training Fit

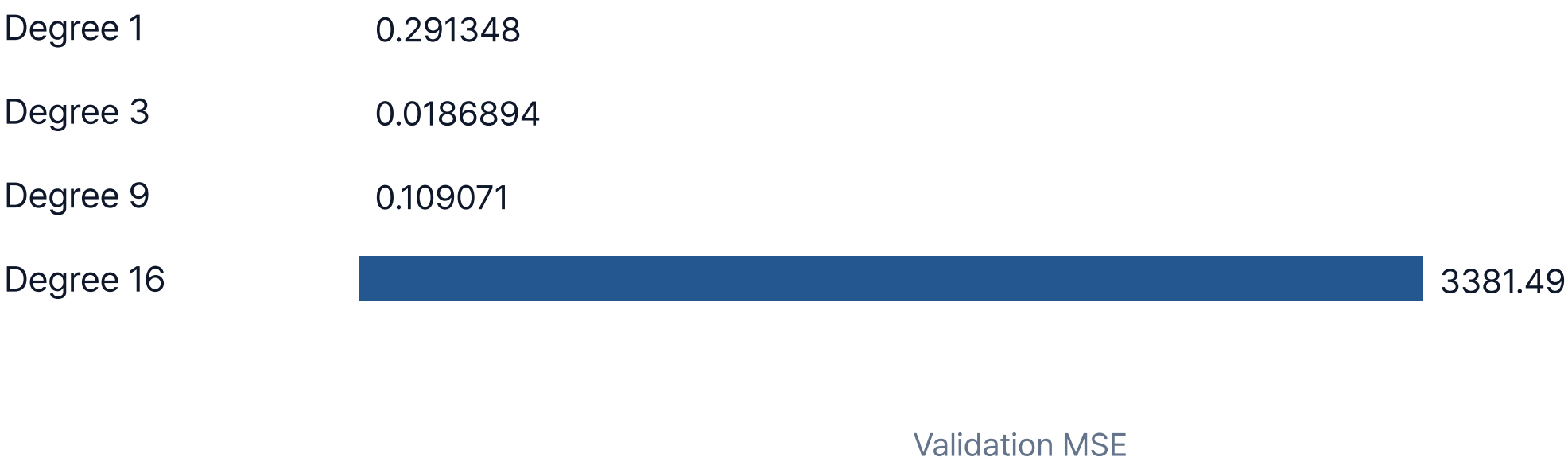
More flexible models can reduce observed training error



Training error decreases as polynomial capacity increases

Capacity and Validation

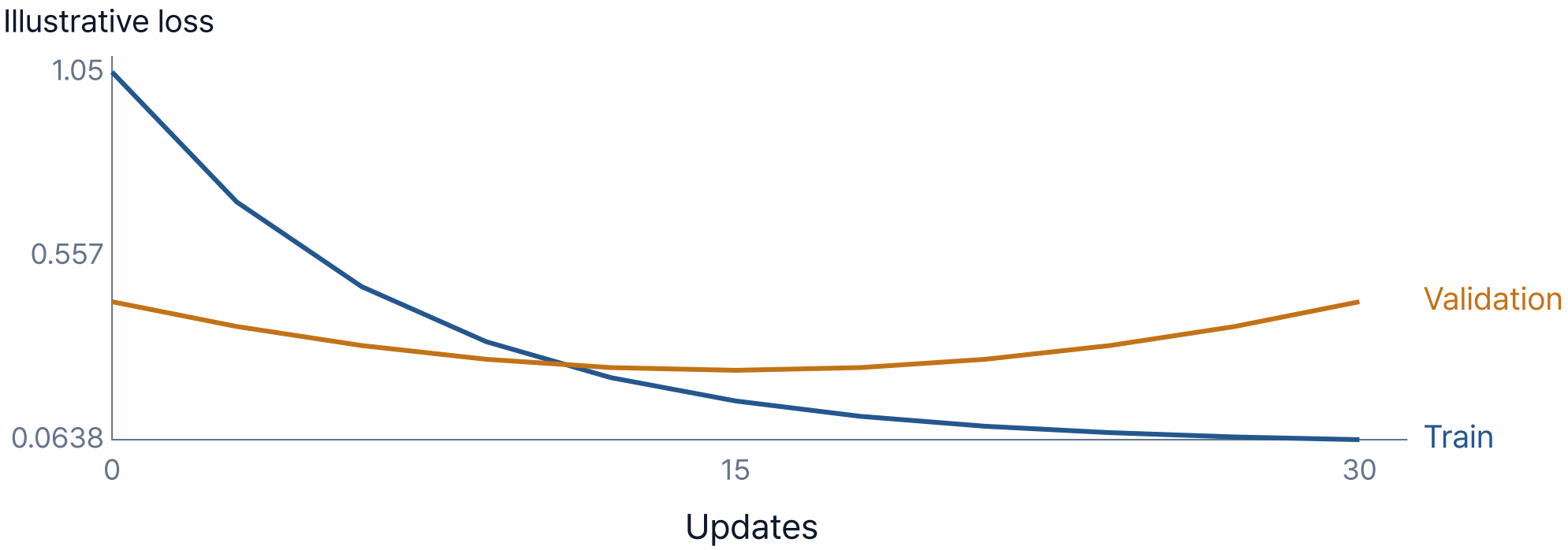
Better training fit can coexist with worse held-out error



The held-out loss distinguishes flexible fits

Learning Curves

Track fit and held-out behavior over update steps



Validation can turn upward while training continues downward

Early Stopping

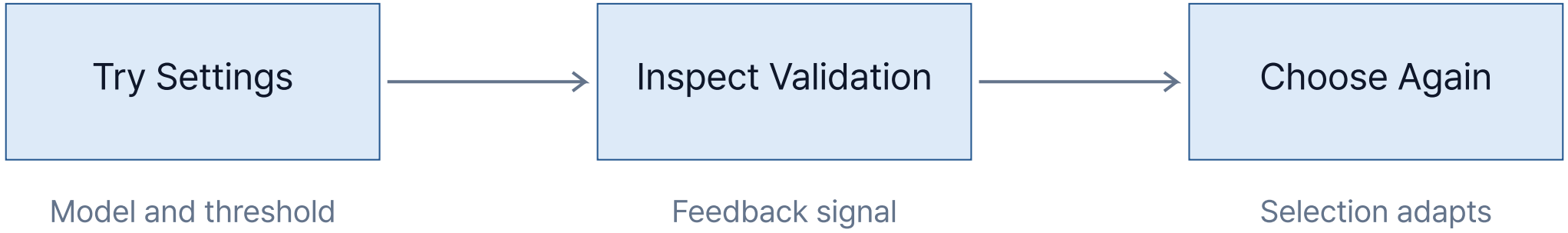
Select a saved state before the full training budget ends

Epoch	Validation Loss
5	0.45
10	0.31
15	0.25
20	0.29

The selected state is epoch fifteen in this example

Caveat: Validation Reuse

Repeated choices can adapt to the validation set



The validation set becomes part of the selection procedure

Data Leakage Diagnosis

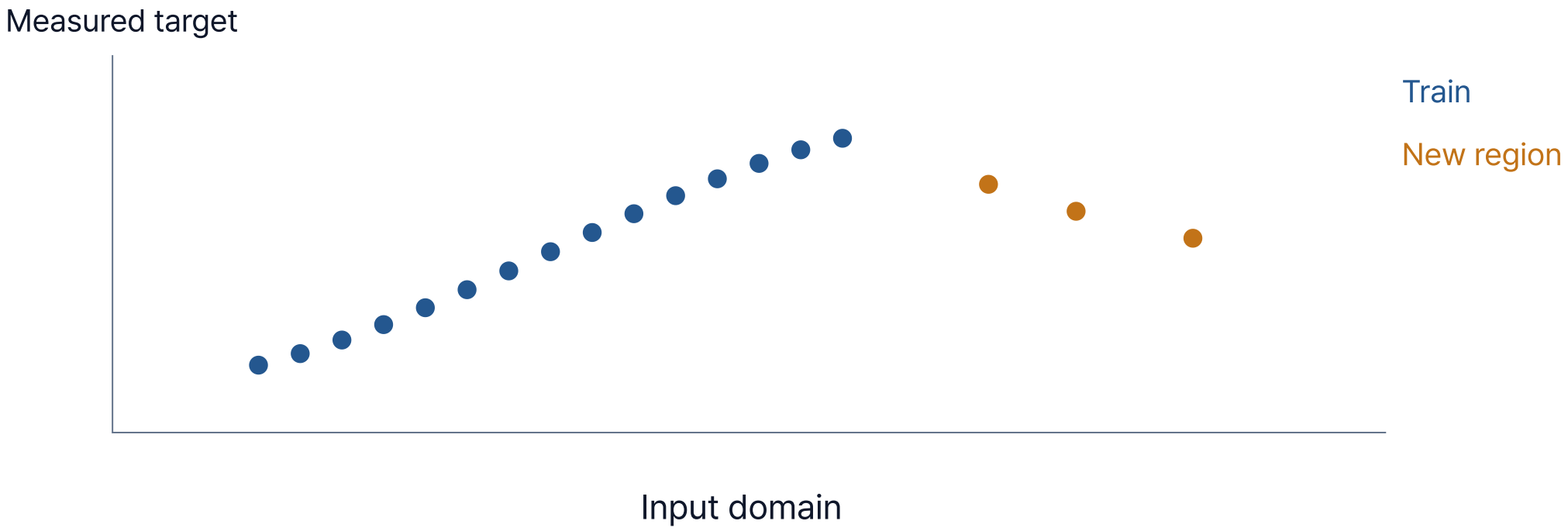
A suspiciously strong score warrants a boundary audit

Leak	Example
Target timing	Future failure status
Entity overlap	Same machine in both splits
Transform fit	Scale on all rows
Selection	Tune using test labels

The split boundary matters before optimizer sophistication

Distribution Shift Evidence

Compare deployment inputs with the training coverage



New operating regions require evidence beyond interpolation

Error Slices

Averages can hide a small failing subgroup

	Cases	Mean Loss
Common	90	0.1
Rare	10	1.0
Overall	100	0.19

A low overall loss can hide ten difficult cases

02 · Optimization Diagnostics

01 Generalization Evidence

Read the data boundary

02 **Optimization Diagnostics**

Control update behavior

03 Regularization and Experiments

Test controlled changes

The Learning Rate

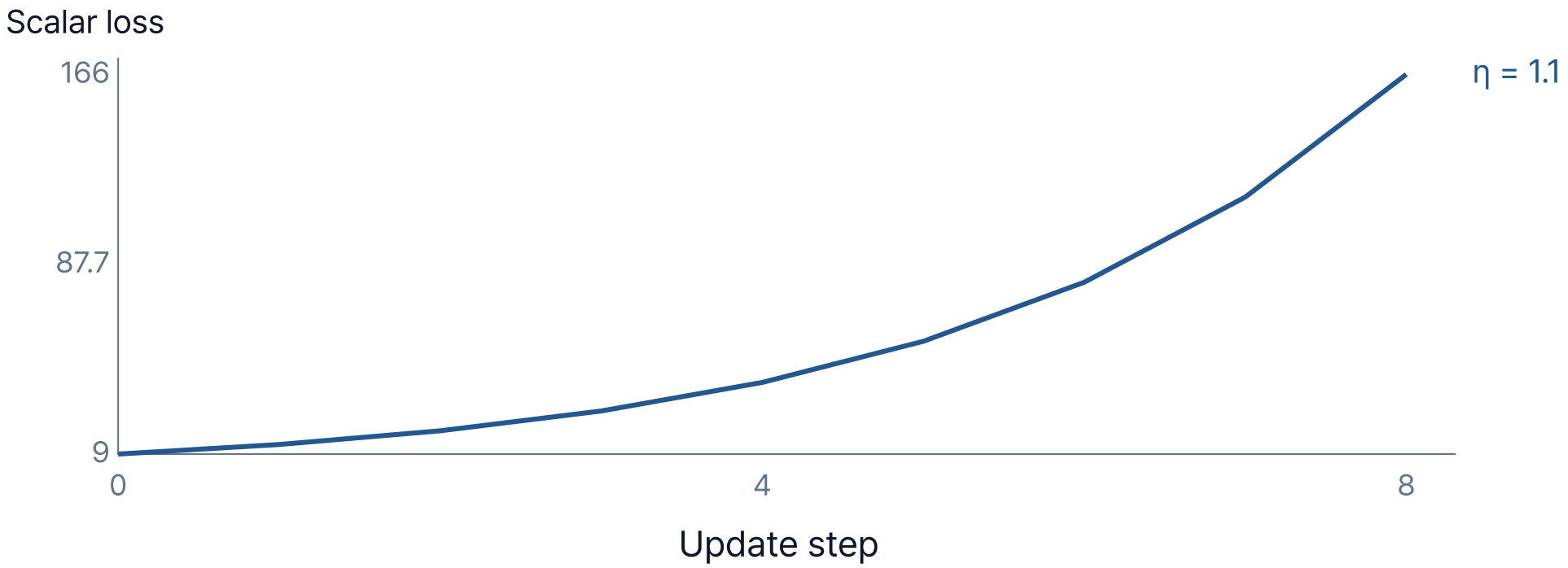
The step size controls how far each gradient moves



Both rates contract this quadratic error

Caveat: Divergent Updates

An accurate gradient can produce growing loss

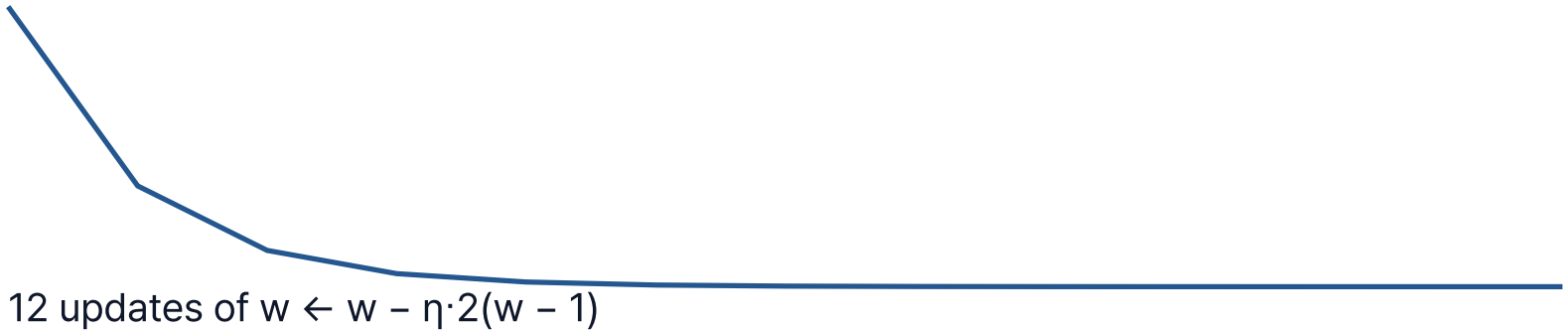


The error grows when the multiplier magnitude exceeds one

Demo: Learning Rate

Locate the stable interval and compare oscillation with divergence

Learning rate 0.2



12 updates of $w \leftarrow w - \eta \cdot 2(w - 1)$

Learning rate 0.20 | w after 12 steps 1.00653 | loss 0.00004265

For this quadratic, $0 < \eta < 1$ converges; $\eta \geq 1$ does not converge from $w = 4$

Answer: The Stability Condition

The scalar error follows a geometric sequence

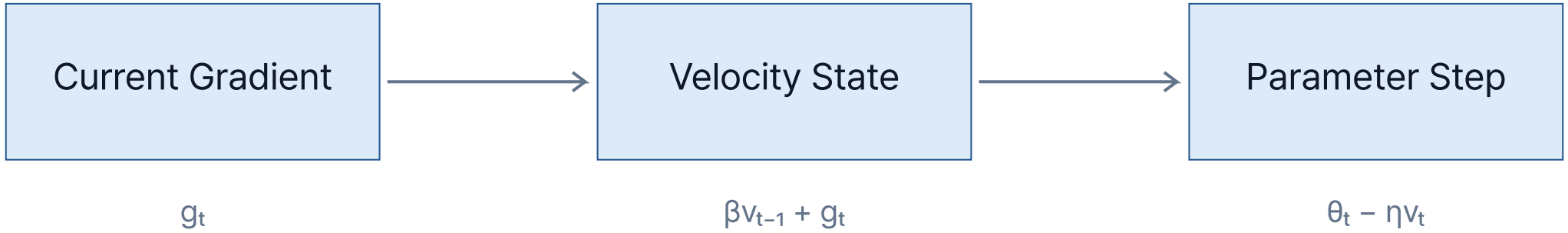
$$e_t = (1 - 2\eta)^t e_0$$

η	Multiplier
0.2	0.6
0.8	-0.6
1.0	-1.0
1.1	-1.2

Oscillation can still converge when the magnitude stays below one

Momentum State

Retain a moving direction across gradient steps



The optimizer carries state beyond the current gradient

Adam Moment Estimates

Track the first and second moments before scaling the update

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = m_t / (1 - \beta_1^t) \quad \hat{v}_t = v_t / (1 - \beta_2^t)$$

$$\theta_t = \theta_{t-1} - \eta \, \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

Both moment estimates are optimizer state

https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html

AdamW Weight Decay

Decay parameters separately from the moment-based gradient update

Gradient path: $g_t \rightarrow m_t, v_t \rightarrow$ adaptive update

Decay path: $\theta \leftarrow (1 - \eta\lambda)\theta$

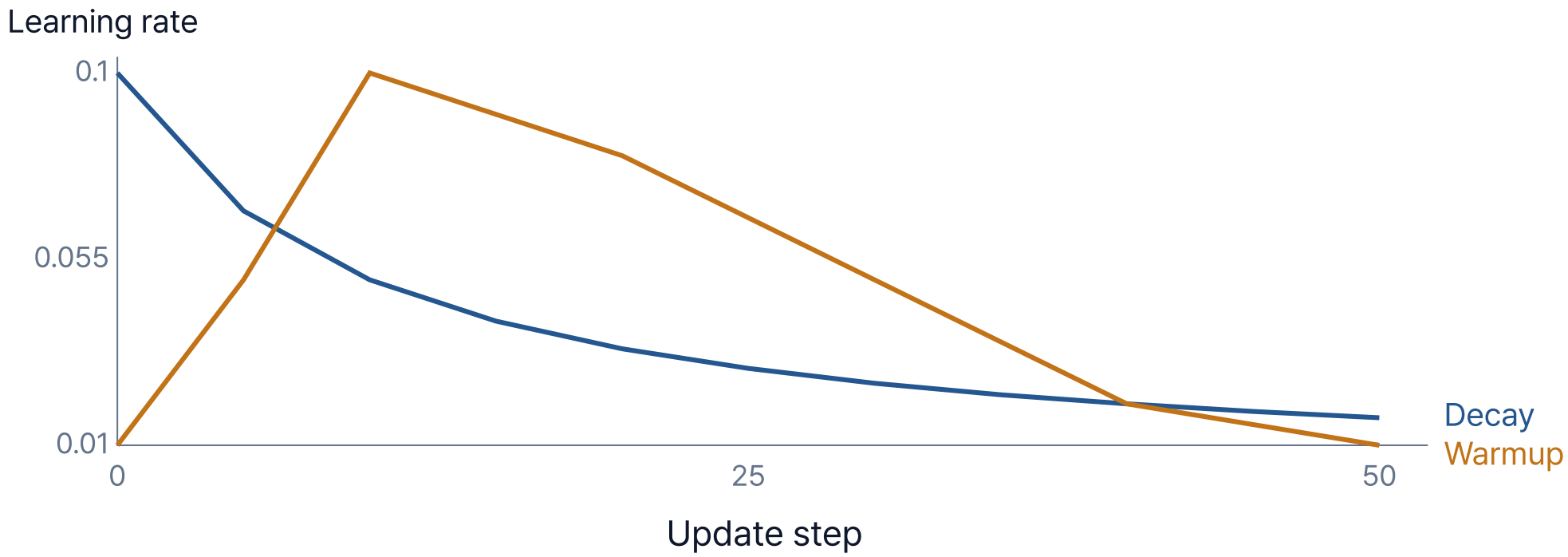
$$\theta_t = (1 - \eta\lambda)\theta_{t-1} - \eta\, \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

Decoupled decay does not enter the moment estimates

https://docs.pytorch.org/docs/stable/generated/torch.optim.AdamW.html

Learning Rate Schedules

Change the update scale during a run



Warmup increases the scale before the later decay

Initialization Scale

Choose weight scale to avoid immediate saturation or explosion



Illustrative activation variance

Initial scale changes the magnitude of forward signals

Gradient Clipping

Limit the size of an exceptional update signal

	g_1	g_2	$\|g\|$
Before	3	4	5
Norm Clipped	1.2	1.6	2

$$g_{clipped} = g \times \min(1, c / \|g\|)$$

Clipping to norm two preserves the gradient direction

Normalization Axes

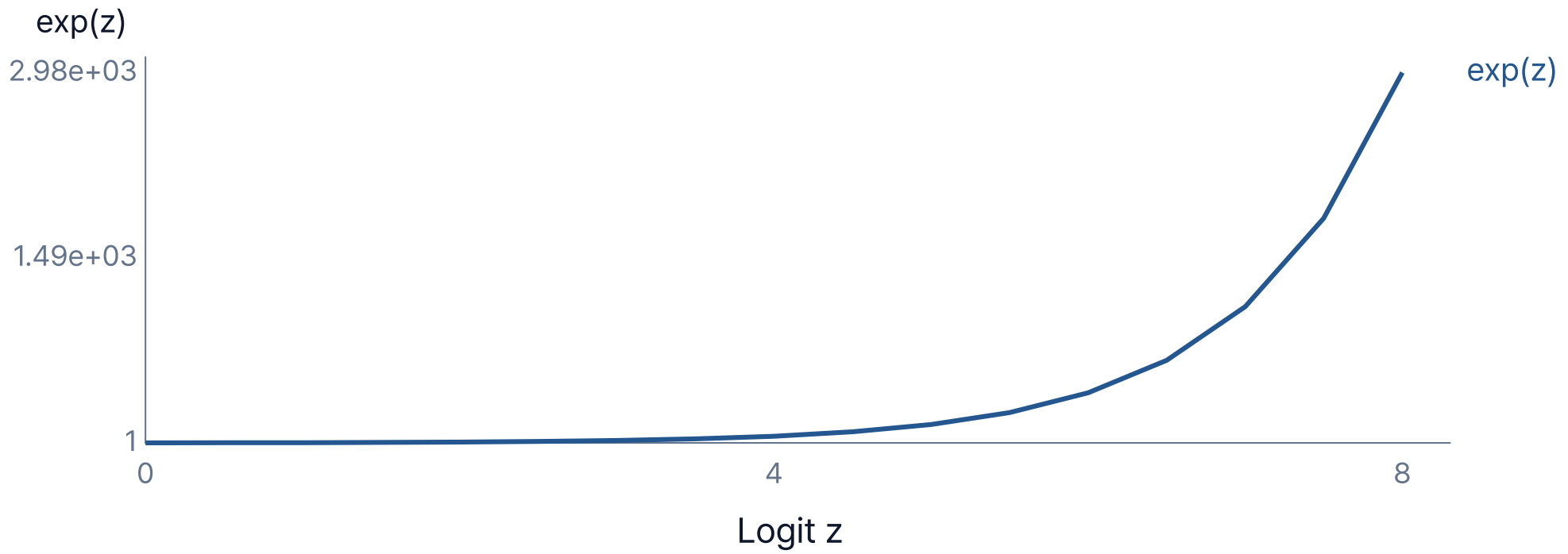
Batch and layer normalization aggregate different dimensions

Statistics Axis	Method
Across observations	BatchNorm
Within one example	LayerNorm

The statistic axis determines which examples interact

Numerical Stability

Preserve the same objective using stable arithmetic



Exponentials grow rapidly even for moderate positive logits

03 · Regularization and Experiments

01 Generalization Evidence

Read the data boundary

02 Optimization Diagnostics

Control update behavior

03 **Regularization and Experiments**

Test controlled changes

Regularization Mechanisms

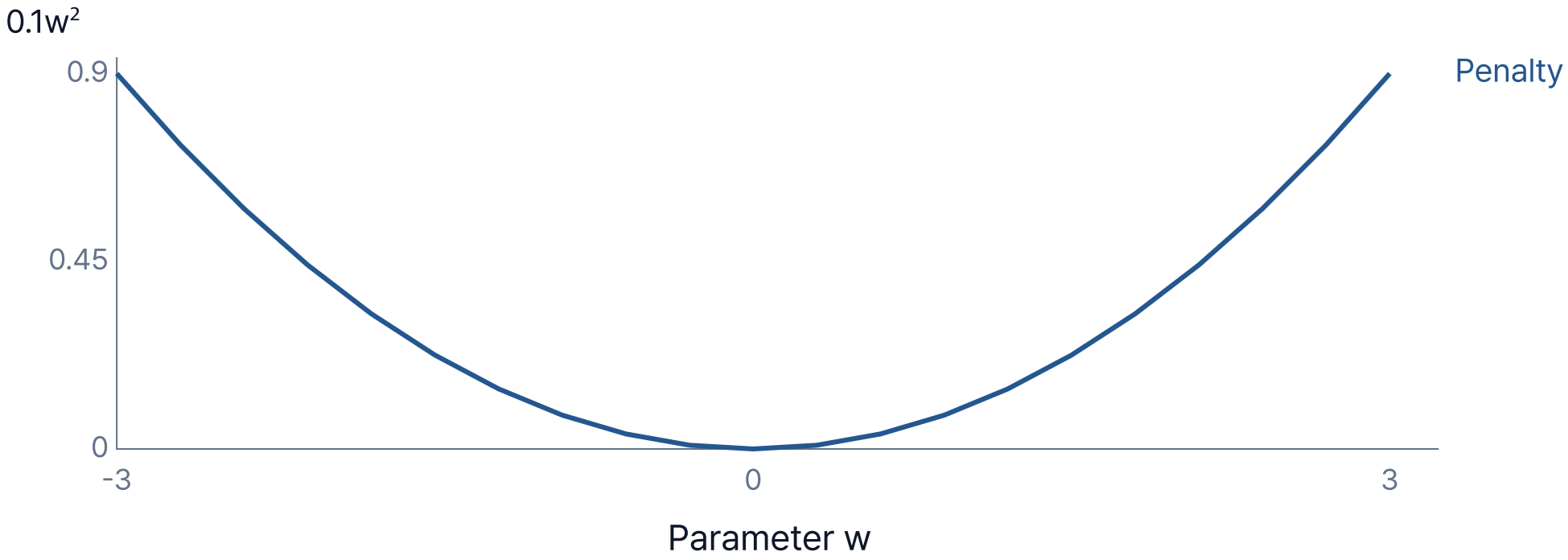
Different controls restrict different failure modes

Control	Primary Effect
Weight decay	Penalize parameter magnitude
Dropout	Randomize hidden activations
Early stopping	Limit fitting duration
Augmentation	Expand chosen input variations

Regularization changes either the objective or the learning experience

Weight Decay and Model Scale

A magnitude penalty changes the optimization tradeoff



Large parameter magnitudes pay a larger penalty

Dropout Masks

Training uses random masks with a scale correction

	h_1	h_2	h_3	h_4
Activation	2	4	6	8
Keep Mask	0	1	0	1
Scaled Output	0	8	0	16

At keep probability one half, retained values double during training

Augmentation Validity

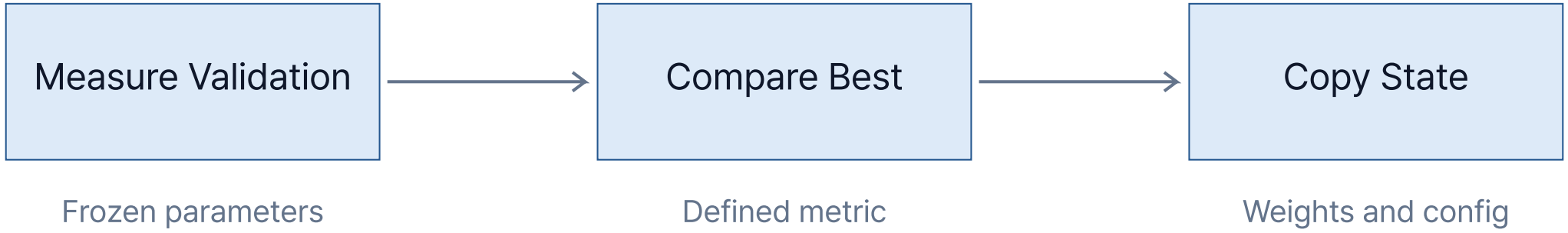
Preserve the task label under the chosen input change

Change	Potentially Valid	Potential Failure
Small image crop	Object retained	Target removed
Text substitution	Meaning retained	Negation changed
Time shift	Event unchanged	Label timing broken

An augmentation is valid only under a task-specific invariance

Checkpoint Selection

Save the best validation state with its configuration



The retained state should correspond to the selected score

Multiple Seeds

Separate an apparent gain from run-to-run variation

	Seed 1	Seed 2	Seed 3
Baseline	0.31	0.29	0.32
Candidate	0.3	0.28	0.31

Compare paired runs before treating a small gap as decisive

Ablation as a Test

Change one component while holding the protocol fixed

Run	Decay	Augmentation
Base	Off	Off
+ Decay	On	Off
+ Augment	Off	On

Separate changes reveal which intervention explains a result

Training Failure Triage

Use the observed failure to choose the next check

Observation	Next Check
Tiny batch cannot fit	Code, loss, gradient
Train good, valid poor	Coverage and capacity
NaN appears	First unstable operation
All classes identical	Targets and imbalance

A diagnostic action follows the nearest visible symptom

Experiment Records

Retain enough evidence to repeat a selected run

Record	Example
Data boundary	Row IDs and split rule
Configuration	Model, optimizer, seed
Evidence	Losses and error slices
Artifact	Selected checkpoint and version

The result includes its data boundary and selected state

Generalization Decision

Use a frozen evaluation to justify the final choice



A useful training run ends with tested deployment assumptions

Takeaway: Training and Generalization

01 Distinguish fitting from generalization evidence

02 Diagnose gradients and numerical scale before adding complexity

03 Retain a selected state and an honest evaluation boundary